

Python Dictionaries

Hash Maps · JSON Foundations · Dictionary Methods

Table of Contents

Part	Topic
1	What is a Dictionary? Why not a List?
2	Hash Map Concept
3	Dictionary Rules (Unique Keys, Immutable Keys, Any Values)
4	Accessing Dictionary Data
5	Adding & Updating Data
6	Nested Dictionaries
7	JSON Foundations
8	Why do we need JSON?
9	Dictionary Methods: keys(), values(), items(), get(), update()
★	Assignment

Day 8 — Class Notes

1 What is a Dictionary? Why Not a List?

A **dictionary** in Python is a built-in data structure that stores data as **key-value pairs**. Every piece of information has a name (key) and a value, just like a real-world dictionary where every word has a definition.

```
# Creating a dictionary
my_dict = {"name": "Dorya", "age": 20}

# Key → Value
# "name" → "Dorya"
# "age" → 20
```

Why not a List?

List	Dictionary
Access by index (position)	Access by meaningful key (name)
student[0] → what is index 0?	student["name"] → clear & readable
Order matters; hard to search	Instant lookup by key (O(1))
No labels — easy to confuse data	Self-documenting structure

■ Key Insight

Use a list when order matters and items are similar.

Use a dictionary when items have names/labels and you need fast lookups.

2

Hash Map Concept

Under the hood, Python dictionaries are implemented as **Hash Maps**. Understanding this helps explain why dictionaries are so fast.

Step 1 — Hashing: When you write `dict['name']`, Python runs the key 'name' through a hash function. This produces a unique integer (hash code).

Step 2 — Bucket Mapping: The hash code is converted into a bucket index. Python stores the value at that memory bucket.

Step 3 — Retrieval: To retrieve, Python hashes the key again, finds the bucket, and returns the value instantly — $O(1)$ time complexity.

```
# Conceptual view of hash map

key = "name"
hash_code = hash(key)      # e.g. 2305843009213
bucket_index = hash_code % table_size
table[bucket_index] = "Dorya"  # stored here

# Python does ALL of this automatically!
```

■ Why is this important?

$O(1)$ average lookup – regardless of dictionary size.

Keys must be hashable → they must be immutable (str, int, tuple).

Lists cannot be keys because they are mutable (unhashable).

3 Dictionary Rules

3.1 — Keys Must Be Unique

A dictionary cannot have two entries with the same key. If you assign a value to an existing key, the old value is simply **overwritten**.

```
contacts = {"Dorya": 1234567890}
contacts["Dorya"] = 9999999999 # overwrites old number
print(contacts["Dorya"])      # 9999999999

# Duplicate key at creation — last one wins
data = {"a": 1, "a": 2}
print(data) # {"a": 2}
```

3.2 — Keys Must Be Immutable

Keys must be **hashable**, which means they must be immutable. Valid key types are strings, integers, floats, and tuples. Lists and other dictionaries **cannot** be keys.

```
# Valid keys
contacts = {
    "Dorya" : 1234567890, # string key ✓
    (1, 2)  : 9876543210, # tuple key ✓
    1       : "One"       # integer key ✓
}

# Invalid key (will raise TypeError)
# bad = {[1, 2]: 'value'} # list key ✗
```

3.3 — Values Can Be Any Data Type

While keys must be immutable, **values have no such restriction**. A value can be a string, number, list, another dictionary, a function — anything.

```
mixed = {
    "name" : "Tanishq", # string value
    "age"  : 21,        # integer value
    "scores" : [90, 85, 92], # list value
    "address": {"city": "Bengaluru"}, # dict value
    "active" : True,     # boolean value
}
```


4

Accessing Dictionary Data

There are two ways to access a value from a dictionary:

Method 1 — Square Bracket Notation []

Directly accesses the value. Raises a **KeyError** if the key does not exist.

```
contacts = {"Dorya": 1234567890, (1,2): 9876543210}

print(contacts["Dorya"])      # 1234567890
print(contacts[(1, 2)])      # 9876543210

# Accessing a missing key raises KeyError
# print(contacts["Tanishq"]) → KeyError: "Tanishq"
```

Method 2 — .get() Method

Safely accesses the value. Returns **None** (or a default) if the key does not exist — no error.

```
print(contacts.get("Dorya"))      # 1234567890
print(contacts.get("Tanishq"))    # None (no crash!)
print(contacts.get("Tanishq", 0)) # 0      (custom default)
```

Feature	[] Bracket	.get()
Missing key	Raises KeyError ✗	Returns None ✓
Custom default	Not possible	<code>get(key, default)</code> ✓
Use when	Key is guaranteed to exist	Key might be absent

5 Adding & Updating Data

Adding a new key-value pair

Simply assign to a new key using bracket notation. Python automatically adds it to the dictionary.

```
contacts = {"Dorya": 1234567890}

# Adding a new entry
contacts["Rohit"] = 1122334455
print(contacts)
# {'Dorya': 1234567890, 'Rohit': 1122334455}

# Works for any key type
name = "Hello"
contacts[name] = 9038590285
```

Updating an existing value

Same syntax as adding — if the key already exists, the value is replaced. Python makes no distinction between add and update with [].

```
contacts["Dorya"] = 9999999999 # update
contacts[(1, 2)] = 1111111111 # update tuple key

# Using .update() to merge another dict
contacts.update({"Alice": 5556667777, "Bob": 1112223333})
```

6

Nested Dictionaries

A **nested dictionary** is a dictionary where one or more values are themselves dictionaries. This is perfect for representing structured records like student data.

```
students_dict = {
    1: {
        "name" : {"first_name": "Dorya", "last_name": "Sharma"},
        "marks": {"Math": 90, "Science": 95, "English": 85}
    },
    2: {
        "name" : "Tanishq",
        "marks": {"Math": 80, "Science": 85, "English": 90}
    }
}

# Access nested value – chain the keys
print(students_dict[1]["marks"]["Science"])    # 95
```

Iterating a Nested Dictionary

```
for student_id, student_info in students_dict.items():
    print(f'Student ID: {student_id}')
    print(f'Name: {student_info["name"]}')
    print('Marks:')
    for subject, marks in student_info["marks"].items():
        print(f' {subject}: {marks}')
```

■ Pattern

Outer loop → each student

Inner loop → each subject's marks

Use `.items()` at every level to get both key and value.

7

JSON Foundations

JSON (JavaScript Object Notation) is a lightweight, text-based format for storing and exchanging structured data. It is the universal language of APIs and web services.

JSON vs Python Dictionary — Side by Side

Python Dictionary	JSON
<code>{"name": "Dorya"}</code>	<code>{"name": "Dorya"}</code>
True / False	true / false
None	null
Single or double quotes for keys	Must use double quotes for keys
Python object in memory	Plain text (string) in a file/network

```
import json

# Python dict → JSON string
student = {"name": "Dorya", "marks": 90, "passed": True}
json_str = json.dumps(student, indent=2)
print(json_str)
# Output:
# {
#   "name": "Dorya",
#   "marks": 90,
#   "passed": true
# }

# JSON string → Python dict
loaded = json.loads(json_str)
print(loaded["name"]) # Dorya
```

8

Why Do We Need JSON?

Python dictionaries live only inside a running Python program. The moment the program stops, the data is gone. JSON solves three major problems:

1. Persistence	Save data to a file and reload it later. JSON files can store user data, settings, databases.
2. Communication	When your Python app talks to a web server, Instagram API, Google Maps — data travels as JSON text.
3. Interoperability	JSON is language-neutral. A Python server can send JSON to a JavaScript frontend, a Java mobile app, or a Go service.

```
import json

# Saving to a file
with open("students.json", "w") as f:
    json.dump(students_dict, f, indent=2)

# Loading from a file
with open("students.json", "r") as f:
    data = json.load(f)
```

9

Dictionary Methods

9.1 — keys()

Returns a view of all keys in the dictionary.

```

contacts = {"Dorya": 1234, "Rohit": 5678}
print(contacts.keys())
# dict_keys(['Dorya', 'Rohit'])

# Convert to a list
all_keys = list(contacts.keys())

```

9.2 — values()

Returns a view of all values in the dictionary.

```

print(contacts.values())
# dict_values([1234, 5678])

# Check if a value exists
print(1234 in contacts.values()) # True

```

9.3 — items()

Returns a view of all key-value pairs as tuples. Most useful for looping.

```

for contact in contacts.items():
    print(type(contact)) # <class 'tuple'>
    print(contact)      # ('Dorya', 1234)

for key, value in contacts.items():
    print(f'{key}: {value}')

```

9.4 — .get() (Safe Access)

Returns the value for a key, or a default if the key is missing.

```
print(contacts.get("Dorya"))      # 1234
print(contacts.get("Unknown"))   # None
print(contacts.get("Unknown", -1)) # -1

# Great for counting or defaulting
score = scores.get("Math", 0) + 10
```

9.5 — .update()

Merges another dictionary (or key-value pairs) into the current dictionary.

```
contacts.update({"Alice": 9090, "Bob": 8080})
print(contacts)
# {'Dorya':1234, 'Rohit':5678, 'Alice':9090, 'Bob':8080}

# Also updates existing keys
contacts.update({"Dorya": 9999}) # overwrites
```